

Estruturas de Dados e Algoritmos 1

Lista de Exercícios — Ponteiros em C

Desenvolver domínio sobre:

- Endereçamento de memória
- Manipulação indireta de variáveis
- Aritmética de ponteiros
- Uso de ponteiros com vetores e strings
- Passagem por referência em funções

Exercício 1 — Inspeção de memória

Escreva um programa que:

- Declare:
 - uma variável inteira
 - uma variável float
 - um ponteiro para cada tipo
- Atribua corretamente os endereços
- Imprima:
 - valor da variável

Exercício 2 — Ponteiros cruzados

Dado:

int A = 10, B = 20;

*int *p1, *p2;*

Faça:

- p1 aponta para A, p2 aponta para B
- Imprima os valores apontados
- Troque os ponteiros (sem trocar os valores das variáveis)
- Imprima novamente

Verifique: O que mudou? O que permaneceu igual?

Exercício 3 — Incremento indireto

- Declare uma variável inteira
- Use um ponteiro para:
 - somar 5
 - multiplicar por 2

- Imprima o resultado final

Atenção: Não acessar diretamente a variável (somente via ponteiro).

Exercício 4 — Swap manual (sem função)

- Declare duas variáveis inteiras
- Use ponteiros para trocar os valores **dentro da main**
- Não use variável auxiliar tradicional (use manipulação via ponteiros)

Exercício 5 — Vetor com ponteiros

Dado um vetor:

```
int v[5] = {10, 20, 30, 40, 50};
```

Faça:

- Percorra usando ponteiro (não use índice)
- Imprima os valores
- Some todos os elementos usando apenas aritmética de ponteiros

Dica: use `*(p + i)` ou incremento de ponteiro

Exercício 6 — Modificação por função

Implemente uma função:

```
void dobra(int *x);
```

- A função deve dobrar o valor de x
- Teste na main

Verifique: o que acontece se passar sem ponteiro?

Exercício 7 — Maior elemento do vetor

Implemente:

```
int maior(int *v, int tamanho);
```

- Use apenas ponteiros (sem `v[i]`)
- Retorne o maior valor

Exercício 8 — Contagem de caracteres

Dada uma string:

```
char nome[] = "programacao";
```

Crie uma função:

```
int conta(char *str, char c);
```

- Retorne quantas vezes c aparece
- Use apenas ponteiros

Exercício 9 — Inversão de vetor (in-place)

Crie uma função:

```
void inverte(int *v, int tamanho);
```

- Inverta os elementos do vetor
- Não use vetor auxiliar
- Use dois ponteiros (início e fim)

Exemplo: [1, 2, 3, 4] → [4, 3, 2, 1]

Exercício 10 — Intercalação de vetores

Crie uma função:

```
void intercala(int *v1, int *v2, int *v3, int n);
```

- Intercale os elementos de v1 e v2 em v3
- Resultado: v3 com tamanho 2n

Exemplo:

```
v1 = [1,3,5]
```

```
v2 = [2,4,6]
```

```
v3 = [1,2,3,4,5,6]
```

☞ **Regra:** usar apenas aritmética de ponteiros

◊ Exercício 11 — Comparação de strings

Implemente sua própria versão de:

```
int compara(char *s1, char *s2);
```

- Retorne:
 - 0 se iguais
 - <0 se $s1 < s2$
 - 0 se $s1 > s2$

Sem usar strcmp

Observações a serem seguidas:

- Evitar uso de indexação ($v[i]$) sempre que possível
- Use as opções abaixo:
 - $*p$
 - $p++$
 - $*(p+i)$
- Atente para a distinção entre:
 - valor
 - endereço
 - conteúdo apontado
 - endereço da variável
 - valor armazenado no ponteiro
 - valor apontado pelo ponteiro